2026 2nd International Conference on Electronic Information, Computer and Aerospace Remote Sensing (EICARS) | 979-8-3195-4907-5/26/$31.00 ©2026 IEEE | DOI: 10.1109/EICARS71536.2026.11694749

An LLM-Driven Intelligent Agent for Adaptive Data Preparation with Strategy Memory

1st Chentian Ju
Department of Complexity Science and Engineering
University of Tokyo
Tokyo, Japan
juct0168@gmail.com

2nd Guobin Zhao
Department of Chemistry
National University of Singapore
Singapore
sxmzhaogb@gmail.com

3nd Yuntong Chen*
School of Mechanical Engineering
Northwestern Polytechnical University
Xi'an, China
*chenyt123@mail.nwpu.edu.cn

***Abstract*—Data preparation, including source profiling, quality diagnosis, and cleaning, remains a labor-intensive bottleneck in data-driven applications. Traditional approaches require manual rule design for each data source and lack adaptability to heterogeneous formats. This paper proposes an LLM-driven intelligent agent for adaptive data preparation. The agent leverages a large language model as its reasoning core to perform three tasks in a closed loop: (1) automatic data source profiling through sampling-based probing, (2) data quality diagnosis and cleaning script generation via LLM code synthesis with sandbox verification, and (3) strategy memory that accumulates validated experiences as structured condition-action-effect triples for reuse. Generated scripts are tested in an isolated environment before deployment, and both successes and failures are recorded to enable continuous improvement without model fine-tuning. This paper presents the architecture design, details the core methods, and discusses the design rationale.**

***Keywords*—Large Language Model, Agent, Data Preparation, Strategy Memory**

I. Introduction

Data preparation encompasses profiling, quality assessment, cleaning, and transformation of raw data, which typically consumes a disproportionate share of effort in data science projects. Data errors arising from inconsistencies, missing values, and format conflicts have long been recognized as a critical challenge across domains [1]. In real-world scenarios, data sources are highly heterogeneous, spanning relational databases, time-series repositories, CSV files, streaming logs, and semi-structured documents, each with distinct schemas, encoding conventions, and quality characteristics. The conventional approach relies on domain experts to manually inspect data samples, define parsing rules, write cleaning scripts, and iteratively refine them. This process is time-consuming, error-prone, and difficult to scale.

Recent advances in large language models (LLMs) have demonstrated remarkable capabilities in code generation [2], natural language understanding, and contextual reasoning. The emerging paradigm of LLM-based agents further extends these capabilities by equipping LLMs with tool invocation, memory management, and autonomous planning, enabling complex multi-step tasks with minimal human intervention. These developments have opened new possibilities for automating data preparation workflows that were previously dominated by manual operations.

However, most existing approaches either focus on isolated data preparation subtasks without end-to-end integration, or adopt complex multi-agent architectures that introduce coordination overhead. Moreover, current agent systems predominantly operate in a stateless manner, where each task is processed independently without leveraging past execution experience, meaning the same type of problem must be solved from scratch every time.

This paper proposes a lightweight, single-agent framework for adaptive data preparation. The core contributions are twofold. First, the agent employs an LLM code synthesis approach with sandbox verification for cleaning script generation: rather than requiring pre-built parsers, the agent analyzes data samples and generates parsing and cleaning scripts automatically, which are verified before deployment. Second, a strategy memory mechanism stores validated processing experiences as structured knowledge, enabling the agent to retrieve and reuse successful strategies and avoid repeating past failures, thereby achieving continuous learning without model weight updates.

The remainder of this paper is organized as follows. Section II reviews related work. Section III presents the proposed framework in detail. Section IV concludes the paper and outlines future directions.

II. Related Work

A. LLMs for Data Preparation

The application of LLMs to data preparation tasks has gained significant traction in recent years. Narayan et al. [3] cast data cleaning and integration tasks as prompting problems and demonstrated that foundation models can generalize effectively to these tasks, achieving state-of-the-art performance even without task-specific fine-tuning. Zhou et al. [4] provided a comprehensive survey covering LLM-based approaches for data cleaning, integration, and enrichment, identifying a fundamental paradigm shift from rule-based, model-specific pipelines toward prompt-driven, context-aware workflows. Their analysis highlighted that LLMs serve as flexible reasoning engines capable of understanding data semantics from context, significantly reducing the need for hand-crafted rules.

Authorized licensed use limited to: University of Arkansas Litte Rock. Downloaded on November 09,2026 at 22:10:08 UTC from IEEE Xplore. Restrictions apply.

Beyond general-purpose LLMs, researchers have also explored domain-specific model tuning for data tasks. Zhang et al. [5] proposed Jellyfish, an instruction-tuned local LLM specifically designed for data preprocessing. By fine-tuning open-source models on curated data preprocessing datasets covering error detection, data imputation, schema matching, and entity matching, Jellyfish achieved performance competitive with GPT-3.5 and GPT-4 while enabling local deployment that ensures data security. This work demonstrated the feasibility of building specialized LLMs for data preparation, though it focused on individual subtasks rather than an integrated agent-based workflow.

In the broader context of automated data pipelines, Trirat et al. [6] proposed AutoML-Agent, a multi-agent LLM framework for full-pipeline automated machine learning covering data retrieval through model deployment. While this framework demonstrated the potential of LLM agents in automating complex data workflows, its multi-agent design introduces coordination complexity that may not be necessary for data preparation tasks alone. These studies collectively confirm the technical feasibility of LLM-driven data preparation but also reveal a gap in lightweight, single-agent solutions that handle the full data preparation lifecycle.

B. LLM-Based Autonomous Agents

The concept of LLM-based autonomous agents has rapidly evolved in recent years. Wang et al. [7] provided the first comprehensive survey of LLM-based autonomous agents, systematically categorizing their key components into planning, memory, and tool use modules, and analyzing how these components interact to enable complex task execution. This survey established a conceptual framework that has guided subsequent agent design.

Several foundational works have established core agent interaction paradigms. Yao et al. [8] proposed ReAct, which interleaves reasoning traces and task-specific actions in LLM outputs, allowing the model to dynamically create and adjust plans while interacting with external environments. Schick et al. [9] introduced Toolformer, a model trained in a self-supervised manner to decide which external APIs to call, when to call them, and how to incorporate results into subsequent predictions. Wei et al. [10] demonstrated that chain-of-thought prompting can elicit step-by-step reasoning in large language models, providing the cognitive foundation for complex agent decision-making.

In the direction of agent self-improvement, Shinn et al. [11] proposed Reflexion, a framework that reinforces language agents through verbal feedback rather than weight updates. Reflexion agents maintain an episodic memory buffer of self-reflective text to guide improved decision-making in subsequent attempts. Madaan et al. [12] proposed Self-Refine, which enables LLMs to iteratively refine their own outputs through self-generated feedback without additional training data. Wang et al. [13] introduced Voyager, an open-ended embodied agent that builds a persistent skill library of executable code, progressively accumulating complex capabilities over time. These works demonstrate that agents can achieve continuous improvement through structured experience accumulation, a principle that our strategy memory mechanism extends to the data preparation domain.

C. Memory and Retrieval Mechanisms for Agents

Equipping agents with external knowledge retrieval and persistent memory is an active research direction. Lewis et al. [14] proposed Retrieval-Augmented Generation (RAG), which grounds LLM outputs in retrieved external knowledge, providing a general mechanism for reducing hallucination and improving factual accuracy. This paradigm has been widely adopted as a foundational component in agent architectures. Krishnan et al. [15] proposed ActiveClean, which integrates data cleaning with statistical model training through a progressive and iterative process and prioritizes records expected to have the greatest impact on model performance.

However, applying structured strategy memory specifically to data preparation, where validated cleaning rules, parsing configurations, and diagnostic patterns can be systematically accumulated and reused, remains underexplored. Most existing memory mechanisms are designed for general-purpose knowledge retrieval or conversational context management, rather than for capturing and reusing domain-specific operational experience. Our work addresses this gap by designing a lightweight strategy memory tailored for data preparation tasks, where each entry captures the full context of a validated processing operation as a structured condition-action-effect triple.

III. Prepare Your Paper Before Styling

A. Overall Architecture

The framework consists of four components operating in a closed loop, as illustrated in Figure 1.

(1) A Control Loop that receives user tasks, orchestrates the workflow, and manages execution state.

(2) A Data Profiling Module that automatically detects data source types and extracts schema information through sampling-based probing.

(3) A Quality Diagnosis and Cleaning Module that identifies data quality issues and generates executable cleaning scripts via LLM code synthesis.

(4) A Strategy Memory that persistently stores validated processing experiences for retrieval and reuse.

The LLM serves as the shared reasoning engine across all modules, receiving structured prompts containing task context, data samples, and retrieved historical strategies, and producing analytical results, executable code, or decision recommendations.

The overall workflow proceeds as follows. Upon receiving a data preparation task, the Control Loop first invokes the Data Profiling Module. The profiling results and data samples are then passed to the Quality Diagnosis and Cleaning Module, which generates cleaning scripts. All generated scripts are executed in an isolated sandbox for verification. Validated scripts are registered as formal configurations and their strategies are stored in Strategy Memory. Failed attempts trigger error analysis and retry with corrected scripts, and failure records are also stored to prevent recurrence.

Authorized licensed use limited to: University of Arkansas Litte Rock. Downloaded on November 09,2026 at 22:10:08 UTC from IEEE Xplore. Restrictions apply.

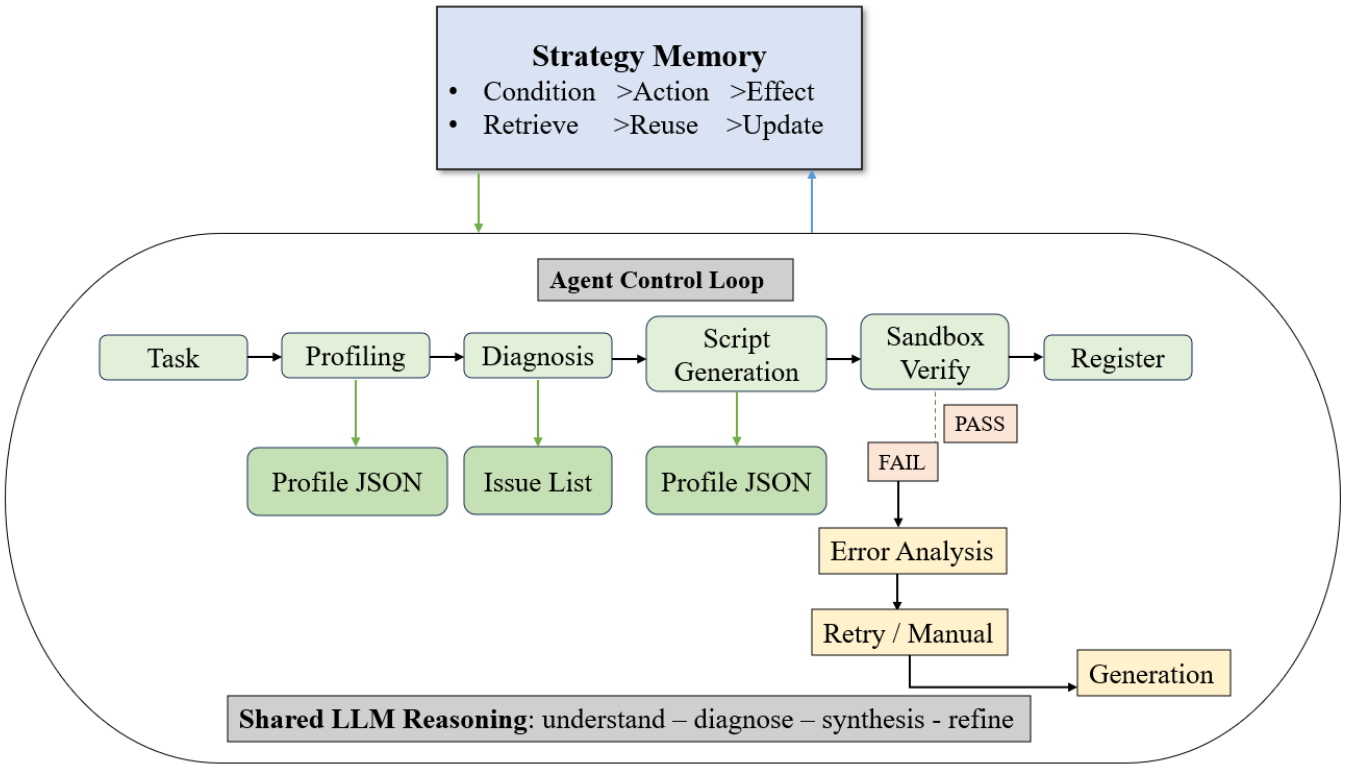


Figure 1: Overall architecture and closed-loop workflow of the proposed framework.

B. *Sampling-Based Data Profiling*

Given a target data location (file path, database endpoint, or streaming source), the Data Profiling Module first extracts a representative sample by reading an initial data fragment. This sample, together with any available metadata, is submitted to the LLM via a structured prompt requesting identification of three aspects: (1) data source type (e.g., relational table, CSV, JSON, time-series stream), (2) field-level semantics (e.g., timestamp, identifier, numeric measurement), and (3) format characteristics (encoding, delimiters, nested structures).

The LLM returns a structured profiling report in a predefined JSON schema. This report is then validated against the actual sample through automated consistency checks, including verifying that identified delimiters correctly split rows, that inferred data types match parsed values, and that the detected encoding successfully decodes the content. If inconsistencies are found, the module re-prompts the LLM with error details for refinement. Before generating the profiling prompt, the module also queries Strategy Memory for any previously stored profiles of similar data sources, injecting them as reference examples to improve recognition accuracy.

C. *LLM Code Synthesis for Adaptive Cleaning*

This module addresses two sequential tasks: quality diagnosis and cleaning script generation.

Quality Diagnosis. Upon receiving profiled data, the module performs statistical analysis to compute quality metrics including null value rates, duplicate ratios, value distributions, and outlier detection results. These metrics, combined with data samples, are provided to the LLM, which produces a structured quality issue list identifying specific problems such as missing values, format inconsistencies, semantic conflicts, and distributional anomalies, along with severity assessments.

Cleaning Script Generation. Based on the quality issue list, the module follows a generate-verify-register pipeline:

Step 1 (Generate): The agent constructs a prompt containing the data sample, profiling report, quality issue list, and relevant historical strategies retrieved from Strategy Memory. The LLM generates a Python cleaning script targeting the identified issues.

Step 2 (Verify): The script is executed in an isolated sandbox against the data sample. Results are validated by checking output completeness, type consistency, and constraint satisfaction.

Step 3 (Register or Retry): If validation passes, the script is registered as a formal cleaning configuration, and the strategy is stored in Strategy Memory. If validation fails, the error message and failed output are fed back to the LLM for correction. A maximum retry limit (default: 3) is enforced; tasks exceeding this limit are flagged for manual intervention, and the failure case is recorded in memory.

This approach eliminates the need for pre-built cleaning rules for each data format. The LLM adapts its generated code based on contextual information from profiling and historical strategies. Sandbox verification ensures that no unvalidated code reaches production data.

D. *Strategy Memory*

Strategy Memory enables the agent to learn from past executions, distinguishing it from stateless LLM invocations where each task starts from scratch.

Authorized licensed use limited to: University of Arkansas Litte Rock. Downloaded on November 09,2026 at 22:10:08 UTC from IEEE Xplore. Restrictions apply.

Representation. Each entry is stored as a structured triple: (Condition, Action, Effect). Condition describes the applicable scenario using attributes such as data source type, field characteristics, and quality issue type. Action records the processing approach, including the generated script or rule template. Effect captures the outcome, including success or failure status, validation metrics, and error information. Each entry also maintains metadata such as creation timestamp, invocation count, and cumulative success rate.

Retrieval. When the agent encounters a new task, it retrieves relevant strategies through a two-stage indexing mechanism. The first stage filters by task category (profiling or cleaning) and data source type. The second stage computes similarity between the current task context and candidate conditions using field-level feature matching. The top-k strategies are injected into the LLM prompt as reference examples.

Accumulation. After each task execution, the agent updates Strategy Memory. Successful strategies are stored with their full context. For failures, the agent extracts a concise failure reason and corrective insight, storing them as negative examples. Over time, the agent's performance on recurring data preparation patterns improves through accumulated experience, achieving continuous learning without model fine-tuning.

IV. Conclusion

This paper presented an LLM-driven intelligent agent for adaptive data preparation. The framework integrates sampling-based data profiling, LLM code synthesis with sandbox verification for cleaning script generation, and a strategy memory mechanism for experience accumulation and reuse. The closed-loop design addresses reliability concerns of LLM-generated code, while strategy memory enables continuous improvement through accumulated experience. Future work will focus on experimental evaluation on public data preparation benchmarks and extension to multi-modal data sources. We will conduct quantitative evaluations on multiple real-world datasets to demonstrate measurable improvements, compare our agent against representative LLM-based data preparation baselines, and analyze the influence of accumulated memory triples on both runtime and cleaning accuracy.

References

[1] Z. Abedjan, X. Chu, D. Deng, R. C. Fernandez, I. F. Ilyas, M. Ouzzani, P. Papotti, M. Stonebraker, and N. Tang, "Detecting data errors: Where are we and what needs to be done?" Proc. VLDB Endow., vol. 9, no. 12, pp. 993–1004, 2016.

[2] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, et al., "Evaluating large language models trained on code," arXiv:2107.03374, 2021.

[3] A. Narayan, I. Chami, L. Orr, and C. Ré, "Can foundation models wrangle your data?" Proc. VLDB Endow., vol. 16, no. 4, pp. 738–746, 2022.

[4] W. Zhou, J. Zhou, H. Wang, Z. Li, Q. He, S. Han, et al., "Can LLMs clean up your mess? A survey of application-ready data preparation with LLMs," arXiv:2601.17058, 2026.

[5] H. Zhang, Y. Dong, C. Xiao, and M. Oyamada, "Jellyfish: Instruction-tuning local large language models for data preprocessing," in Proc. 2024 Conf. Empirical Methods Natural Language Processing (EMNLP), 2024, pp. 8754–8782.

[6] P. Trirat, W. Jeong, and S. J. Hwang, "AutoML-Agent: A multi-agent LLM framework for full-pipeline AutoML," in Proc. 42nd Int. Conf. Machine Learning (ICML), vol. 267, 2025, pp. 60099–60146.

[7] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, et al., "A survey on large language model based autonomous agents," Frontiers Comput. Sci., vol. 18, no. 6, Art. no. 186345, 2024.

[8] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in Proc. Int. Conf. Learning Representations (ICLR), 2023.

[9] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, et al., "Toolformer: Language models can teach themselves to use tools," in Advances in Neural Information Processing Systems, vol. 36, 2023.

[10] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in Advances in Neural Information Processing Systems, vol. 35, pp. 24824–24837, 2022.

[11] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in Advances in Neural Information Processing Systems, vol. 36, 2023.

[12] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, et al., "Self-Refine: Iterative refinement with self-feedback," in Advances in Neural Information Processing Systems, vol. 36, pp. 46534–46594, 2023.

[13] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, et al., "Voyager: An open-ended embodied agent with large language models," arXiv:2305.16291, 2023.

[14] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Advances in Neural Information Processing Systems, vol. 33, pp. 9459–9474, 2020.

[15] S. Krishnan, J. Wang, E. Wu, M. J. Franklin, and K. Goldberg, "ActiveClean: Interactive data cleaning for statistical modeling," Proc. VLDB Endow., vol. 9, no. 12, pp. 948–959, 2016.

Authorized licensed use limited to: University of Arkansas Litte Rock. Downloaded on November 09,2026 at 22:10:08 UTC from IEEE Xplore. Restrictions apply.